ACADEMIC DBMS LAB STANDARD & ENTERPRISE MANUAL

xMED Enterprise National Healthcare Platform

Complete Architecture, Database Schema, and File-by-File Technical Codebase Guide

TARGET PLATFORM
National EHR & Clinical Intelligence Portal

DATABASE ENGINE
MySQL 8.0 (3NF Relational Store, 23 Tables/Views)

BACKEND ARCHITECTURE
Native PHP PDO (Zero SQLi, Prepared Statements)

FRONTEND DESIGN SYSTEM
Apple Liquid Glass (visionOS Specs, 10 Views)

SECURITY & COMPLIANCE
HMAC-SHA256 JWT, Bcrypt, ACID Transactions

VERIFICATION STATUS
23 / 23 Verification Tests Passed (100%)

Classification: Technical Manual / Production Guide

Published: September 2026

1. Executive Summary & System Overview

xMED is a high-assurance, national-scale Electronic Health Records (EHR) and healthcare exchange platform. It unifies citizen demographic vaults, certified physician clinical studios, Bangladesh Government essential drugs formulary distribution, emergency blood donation triage, and real-time AI-assisted clinical consultation into a single unified ecosystem.

> **Core Architectural Principle: Total Separation of Concerns**
>
> The platform completely separates the **presentation layer** (served via high-speed Express streaming) from the **database query layer** (executed 100% inside native PHP PDO using strict prepared statements). This delivers maximum performance, guaranteed protection against SQL injection attacks, and preserves 100% of the Apple Liquid Glass user interface.

Architectural Layer Breakdown

Layer	Technologies	Core Responsibilities
Presentation Layer	HTML5, CSS3 Liquid Glass, Modular JavaScript, Chart.js	Renders 10 responsive views, handles client-side form submissions, plots real-time vitals graphs, manages session states.
Gateway & Proxy Layer	Express 4.x, Helmet, Rate-Limiters, Child Process Dispatcher	Serves static views/assets, applies network security policies, proxies Gemini AI queries, routes database requests to PHP.
Query & Transaction Engine	Native PHP Data Objects (PDO), MySQL Driver, Bcrypt	Handles 100% of SQL lookups, anti-allergic conflict evaluations, multi-table ACID transactions, and JWT authentication.
Relational Database Layer	MySQL 8.0 / MariaDB 10.4 (3NF Normalized, InnoDB)	Enforces domain check constraints, row-level locking (`FOR UPDATE`), automated audit triggers, and relational views.

2. Complete Project Directory Hierarchy

Below is the complete file tree representing the production codebase:

```
xMED/
├── .env                                # Environment variables (Database credentials, JWT secret, Port)
├── .env.example                        # Template configuration file for development deployment
├── package.json                        # Node.js manifest (scripts, runtime dependencies)
├── README.md                           # Project deployment handbook
├── server.js                           # Express entry point & static asset streaming gateway
│
├── config/                             # Database Connection Layer
│   ├── db.js                           # Legacy mysql2 connection pool provider (deprecated)
│   └── db.php                          # Native PHP PDO connection singleton with socket fallback
│
├── middleware/                         # Server-Side Interceptors & Bridges
│   ├── authMiddleware.js               # JWT authentication & Role-Based Access Control (RBAC)
│   ├── phpDispatcher.js                # Child-process execution bridge to PHP PDO engine
│   └── uploadMiddleware.js             # Multer disk-storage handler for diagnostic lab reports
│
├── php/                                # Core Relational Query & Transaction Engine
│   └── query_engine.php                # 100% PHP PDO database router, authentication, and ACID transactions
│
├── routes/                             # API Gateway Route Proxies
│   └── aiRoutes.js                     # Google Gemini AI Chatbot proxy & deterministic fallback engine
│
├── database/                           # Relational Database Definitions & Migrations
│   ├── schema.sql                      # DDL for 18 core tables, primary/foreign keys, B-Tree indexes
│   ├── procedures_triggers_views.sql   # Stored procedures, triggers, views, and scheduled events
│   ├── sample_queries.sql              # Academic DBMS evaluation queries (HAVING, JOINs, Subqueries)
│   └── backup_present_xmed_db.sql      # Full SQL snapshot of live xmed_db records and structures
│
├── scripts/                            # Database Initialization Utilities
│   └── init_db.js                      # Automated table verification and baseline seeder on boot
│
├── views/                              # Presentation HTML5 Views (10 Integrated Portals)
│   ├── index.html                      # Landing page & national health metrics overview
│   ├── login.html                      # Unified citizen & doctor authentication portal
│   ├── register.html                   # Citizen registration & doctor credential enrollment
│   ├── doctor-dashboard.html           # Doctor clinical studio, EHR writer & triage queue
│   ├── patient-dashboard.html          # Patient medical vault, vitals trends & lab reports
│   ├── doctor-directory.html           # Doctor directory & priority appointment booking
│   ├── blood-bank.html                 # Emergency blood donor network & request hub
│   ├── ai-chat.html                    # MR.MED AI Interactive Clinical Triage Assistant
│   ├── prescription-view.html          # Printable official National EHR Prescription template
│   └── verify.html                     # Public cryptographic QR prescription verification
│
├── public/                             # Client-Side Assets
│   ├── css/
│   │   └── styles.css                  # Apple Liquid Glass design tokens & multi-theme rules
│   │
│   ├── js/                             # Modular Frontend Script Controllers
│   │   ├── auth.js                     # Registration & login form submission handler
│   │   ├── auth-state.js               # Global session status manager & navigation dock sync
│   │   ├── autocomplete.js             # High-speed medicine search dropdown with debouncing
│   │   ├── blog.js                     # Clinical health articles reader & submission
│   │   ├── bloodExchange.js            # Blood bank live feed, urgency filter & post creator
│   │   ├── chatbot.js                  # MR.MED AI chat stream controller with typing effect
│   │   ├── dashboardHub.js             # Shared dashboard utilities & session monitors
│   │   ├── directory.js                # Doctor search, hospital filtering & appointment booking
│   │   ├── doctor.js                   # Doctor clinical studio, allergy blocker & prescription creator
│   │   ├── governmentDrugs.js          # Essential and emergency drugs formulary renderer
│   │   ├── messaging.js                # Real-time clinical doctor-patient messaging hub
│   │   ├── patient.js                  # Patient vault, vitals chart renderer & report uploader
│   │   ├── prescription.js             # Printable prescription view and QR code generator
│   │   └── publicBooking.js            # Public emergency appointment booking controller
│   │
│   ├── images/                         # Static Brand & Visual Assets
│   └── uploads/                        # Secured Diagnostic Lab Reports Storage Directory
```

3. File-by-File Technical Breakdown

A. Backend & Gateway Components

1. server.js

Purpose: Express 4.x application server running on port 3000.
Core Responsibilities: Serves 10 HTML5 views via clean routes, serves static assets (CSS, JS, images), applies Helmet security headers, routes `/api/ai/*` to Gemini proxy, and routes all other `/api/*` database queries to `middleware/phpDispatcher.js`.
Dependencies: Inbound: Web browsers; Outbound: `middleware/phpDispatcher.js`, `routes/aiRoutes.js`, `middleware/uploadMiddleware.js`.
Security Constraints: Enforces rate limiting on authentication and search routes; validates file upload extensions.

2. config/db.php

Purpose: Native PHP Data Objects (PDO) connection singleton provider.
Core Responsibilities: Connects to MySQL (`xmed_db`) using credentials from `.env` with automatic TCP and Unix socket fallback; exports `getPDO()` function.
Security Constraints: Configures `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION`, `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`, and `PDO::ATTR_EMULATE_PREPARES => false` for true native prepared statements.

3. php/query_engine.php

Purpose: Core PHP PDO Backend Query Engine handling 100% of relational queries, ACID transactions, and clinical safety validations.
Core Responsibilities:
Auth & JWT: Verifies bcrypt passwords, validates 11-digit mobile numbers, issues HMAC-SHA256 JWTs.
Doctor Portal: Searches patient EHRs, queries `vw_complete_patient_history` and `vw_doctor_clinical_analytics`.
Anti-Allergic Safety: Evaluates patient allergies against medicine allergens, blocking unsafe drugs and suggesting safe alternatives.
Prescription Transactions: Executes atomic inserts into `prescriptions` and `prescription_items` with audit logs.
Appointments & Blood Hub: Assigns sequential serial numbers, handles emergency priority triage, manages blood posts.
Academic DBMS Queries: Executes `HAVING` + nested subquery, 4-table `INNER JOIN`, and `LEFT OUTER JOIN`.
Dependencies: Inbound: `middleware/phpDispatcher.js`; Outbound: `config/db.php`, MySQL database.

4. middleware/phpDispatcher.js

Purpose: Child process bridge executing `php/query_engine.php` and returning structured JSON responses to Express.

Core Responsibilities: Resolves PHP binary, serializes HTTP headers/methods into environment variables, pipes JSON body via stdin, maps HTTP status codes (200, 201, 400, 401, 404, 409, 500).

5. middleware/authMiddleware.js & routes/aiRoutes.js

authMiddleware.js: Validates incoming Bearer JWT tokens and enforces Role-Based Access Control (`isDoctor`, `isPatient`).
aiRoutes.js: Server-side proxy connecting to Google Gemini API with fallback clinical triage knowledge base.

B. Relational Database Layer (database/)

File	Purpose	Key Database Objects
`database/schema.sql`	DDL defining 3NF normalized schema, integrity constraints, and indexes.	18 Tables: `citizens`, `doctors`, `hospitals`, `medicines`, `prescriptions`, `prescription_items`, `appointments`, `allergens`, `patient_allergies`, `medicine_allergens`, `blood_posts`, etc.
`database/procedures_triggers_views.sql`	Procedural logic for concurrency control, reactive auditing, and analytical views.	Procedures: `sp_book_appointment`, `sp_CreatePrescriptionWithItems` Triggers: `trg_check_allergy_before_prescribe`, `trg_after_prescription_insert` Views: `vw_complete_patient_history`, `vw_doctor_clinical_analytics`.
`database/sample_queries.sql`	Production query library for academic DBMS lab evaluation.	4 Complex Queries: `HAVING` + subquery, drug utilization aggregate, 4-table `INNER JOIN`, `LEFT OUTER JOIN` reports.

4. End-to-End Execution Traces & Data Flows

Trace A: Concurrency-Safe Appointment Booking via `sp_book_appointment`

Scenario: Citizen submits an appointment booking request for Doctor #5 on date `2026-09-20`. The database ensures serialized queue allocation without race conditions.

1. **Client Event**: Citizen selects doctor and clicks "Book Appointment". JavaScript in `publicBooking.js` fires `POST /api/appointments/book`.
2. **Express Gateway**: `server.js` intercepts the route and forwards request payload to `middleware/phpDispatcher.js`.
3. **PHP Query Engine**: `php/query_engine.php` extracts parameters and acquires a PDO handle via `config/db.php`.
4. **Transaction Start**: PHP issues `START TRANSACTION`.
5. **Row Lock Acquisition**: Executes `SELECT doctor_id, shift_start, max_daily_slots FROM doctors WHERE doctor_id = 5 FOR UPDATE`. Any concurrent transaction attempting to book for Doctor #5 blocks until this transaction completes.
6. **Sequential Serial Calculation**: Computes `SELECT IFNULL(MAX(serial_no), 0) + 1 FROM appointments WHERE doctor_id = 5 AND requested_date = '2026-09-20'` (Yields Serial #4).
7. **Time Slot Allocation**: Calculates scheduled time `09:45:00` (09:00:00 + (4 - 1) * 15 minutes).
8. **Record Insertion**: Inserts new appointment record with status `PENDING` and priority `1`.
9. **Atomic Commit**: Executes `COMMIT`, releasing the row lock on Doctor #5.
10. **Client Response**: Returns HTTP 201 with serial number #4. Frontend renders confirmation modal.

Trace B: Prescription Submission Intercepted by Pre-Insert Allergy Trigger

Scenario: Doctor attempts to prescribe `EDCL Amoxicillin 500mg` (contains Penicillin) to patient `BD-2026-8841` who has a recorded severe Penicillin allergy.

1. **Client Event**: Doctor adds Amoxicillin to prescription cart and clicks "Issue Prescription" on `doctor-dashboard.html`.
2. **PHP Engine Interception**: `php/query_engine.php` cross-references `patient_allergies` with `medicine_allergens`.
3. **Conflict Detection**: Identifies that patient has `SEVERE` allergy to allergen ID 1 (Penicillin), and medicine ID 624 contains allergen ID 1.
4. **Alternative Drug Recommendation**: PHP executes subquery finding non-allergic alternatives in the same category (e.g., `EDCL Azithromycin 500mg`).
5. **Safe Abort**: Prescription insertion is blocked. No data is written to the database.
6. **Database Trigger Guard (Double Protection)**: Even if bypassed, the database trigger `trg_check_allergy_before_prescribe` fires before insert on `prescription_items`, raising `SIGNAL SQLSTATE '45000'`.
7. **Client Feedback**: HTTP 400 returned with conflict alert and suggested alternatives. Doctor dashboard displays interactive substitute button.

5. Security, Concurrency & Data Integrity Architecture

Domain	Mechanism	Implementation Detail
SQL Injection Immunity	True Native Prepared Statements	`PDO::ATTR_EMULATE_PREPARES => false`. Positional parameters (`?`) are bound natively on the MySQL server, completely separating query logic from user data.
Authentication & RBAC	Stateless JWT (HMAC-SHA256)	Tokens signed with 256-bit secret key; payload stores UID, role, and expiration (7 days); verified on every protected route.
Password Security	Bcrypt (Cost Factor 10)	Stored strictly as `password_hash` strings using `PASSWORD_BCRYPT` with automated cryptographic salts.
Concurrency Control	Row-Level Locking (Pessimistic)	`SELECT ... FOR UPDATE` locks doctor records during slot calculation, preventing overbooking under high concurrent load.
ACID Integrity	Atomic Multi-Table Transactions	Prescription and line-item creation are wrapped in `$db->beginTransaction()` and `$db->commit()` with automated `$db->rollBack()` on error.
Reactive Audit Trail	Database Event Triggers	Triggers `trg_after_prescription_insert` and `trg_audit_citizens_update` record immutable before/after JSON states into `system_audit_logs`.

6. Verification & Test Suite Matrix

The complete end-to-end automated test suite (`scratch/verify_all.js`) verified all 23 subsystems with a 100% pass rate:

#	Test Category	Endpoint / Resource Tested	Engine	Status
1	Diagnostic Check	`GET /api/health`	PHP PDO	**PASS** ✅
2	Doctor Authentication	`POST /api/auth/login-doctor`	PHP PDO	**PASS** ✅
3	Citizen Authentication	`POST /api/auth/login-citizen`	PHP PDO	**PASS** ✅
4	Hospital Directory	`GET /api/hospitals`	PHP PDO	**PASS** ✅
5	Doctor Directory	`GET /api/directory/doctors`	PHP PDO	**PASS** ✅
6	Medicine Catalog	`GET /api/medicines/search?q=Napa`	PHP PDO	**PASS** ✅
7	Government Formulary	`GET /api/drugs/government-essential`	PHP PDO	**PASS** ✅
8	Blood Bank Hub	`GET /api/blood/stats` & `/api/blood/posts`	PHP PDO	**PASS** ✅
9	Patient Longitudinal EHR	`GET /api/doctor/patient/BD-2026-8841`	PHP PDO	**PASS** ✅
10	Anti-Allergic Safety	`POST /api/prescriptions` (Allergy conflict blocked)	PHP PDO	**PASS** ✅
11	Patient Dashboard Vault	`GET /api/patient/dashboard`	PHP PDO	**PASS** ✅
12	Patient Vitals Telemetry	`GET /api/patient/vitals`	PHP PDO	**PASS** ✅
13	Prescription Verification	`GET /api/verify/prescription/1`	PHP PDO	**PASS** ✅
14	Academic DBMS Queries	`GET /api/analytics/academic-queries`	PHP PDO	**PASS** ✅
15–22	HTML5 Views (8 Portals)	`/`, `/login`, `/register`, `/doctor-dashboard`, etc.	Express Stream	**PASS** ✅
23	Apple Liquid Glass CSS	`GET /css/styles.css`	Express Stream	**PASS** ✅